

L.E.X.I.S

LEGO Expert Identification System

[Abstract](#)

AI-powered system that performs real-time LEGO part detection, classification, and color recognition using computer vision techniques.

Contents

L.E.X.I.S LEGO Expert Identification System	3
1. Project Summary	3
2. Objectives	3
3. Core Features	3
4. Model Structure	4
5. Workflow Pipeline	5
6. Innovation Highlights.....	5
7. Use Cases	5
8. Challenges Addressed	6
CNN Training Code for L.E.X.I.S	10
1. Imports.....	10
2. Configuration & Model Setup	10
3. Dataset Loading & Augmentation	11
4. Model Architecture (MobileNetV2 + Custom Head)	12
5. Compilation & Callbacks	12
6. Handling Class Imbalance.....	13
7. Model Training.....	14
8. Save the Model.....	14
9. Plot Training Curves	14
10. Evaluate the Model	15
CNN Classifiers for L.E.X.I.S.....	17
1. CNN23 – MobileNetV2 Classifier	17
2. CNN7 – Ambiguity Resolver Classifier	19
3. Ensemble Usage in LEXIS	19
YOLO model for L.E.X.I.S	22
1. Model Details.....	22
3. Inference Pipeline (Standalone).....	23
4. Filtering and Postprocessing	23
5. Integration with LEXIS.....	24
YOLO-CNN Hybrid method Code for L.E.X.I.S	29
1. Imports & Setup	29
2. Configuration Parameters	29
3. Model Paths & Loading.....	32
4. Labels & Classification Lists.....	34
5. Color Matching Function	37
6. Image Pre-processing.....	40
2. Normalize Pixel Values	41
3. Expand Dimensions.....	41
7. Patch Classification.....	42
8. Main Video Capture Logic	46
9. Camera and File Classifiers	54
10. Saving Results.....	55

11.	Single Image Classification	56
12.	GUI Interface	56

L.E.X.I.S LEGO Expert Identification System

1. Project Summary

LEXIS is an AI-powered system that performs real-time LEGO part detection, classification, and color recognition using computer vision techniques. It combines:

- ♦ Object detection via YOLO
- ♦ Deep learning classification via MobileNetV2 and CNN7
- ♦ Color matching to LEGO's official palette

Applications include:

- ♦ LEGO sorting
 - ♦ Recycling and reuse
 - ♦ Inventory tracking
 - ♦ Symbol learning and educational tools
-

2. Objectives

LEXIS is built to:

- ♦ Automatically identify LEGO parts in images or live video
 - ♦ Handle small or visually similar parts
 - ♦ Determine the color of each part using BGR matching
 - ♦ Operate via webcam or smartphone stream
 - ♦ Provide accurate predictions using model ensembles
 - ♦ Support home, educational, and industrial use cases
-

3. Core Features

A. YOLO Object Detection

- ♦ Detects LEGO part locations using bounding boxes
- ♦ Lightweight, real-time operation with an optimized `.pt` model
- ♦ Skips problematic classes to reduce misclassification

B. CNN-Based Classification

- MobileNetV2 (CNN23): classifies 23 LEGO parts
- CNN7: focuses on 7 ambiguous classes using synthetic data
- Ensemble logic increases reliability

C. Color Recognition

- Extracts center pixel color from patch
- Matches to closest LEGO color using Euclidean distance
- Outputs both class and color label (e.g., "1x2 Brick - Bright Red")

D. GUI Interface

- Built using Tkinter
- Functions include:
 - Webcam / Phone Camera toggle
 - Image classification
 - Save results to .xlsx
- Real-time visual overlays: boxes, labels, confidence, color

E. Results Logging

- ♦ Logs include:
 - Part name
 - Classification confidence
 - Detected color
 - ♦ Exportable to Excel for analysis or tracking
-

4. Model Structure

1. YOLO (Ultralytics)

- Input: full video frame (e.g. 640x640)
- Output: bounding boxes, class IDs, confidence scores

2. MobileNetV2 (Keras)

- ♦ Input: cropped patch (96x96)
- ♦ Output: 23-class prediction via softmax

3. CNN7 (Keras)

- ♦ Input: same patch
 - ♦ Output: 7-class softmax prediction
 - ♦ Only used when target is ambiguous
-

5. Workflow Pipeline

See detailed breakdown in section [L.E.X.I.S Workflow Pipeline – Detailed Process Description](#).

1. Capture video frame
 2. Detect bounding boxes with YOLO
 - Fallback: center patch if no detection
 3. Pass patch to MobileNetV2
 4. If in ensemble class list, blend with CNN7 output
 5. If in skip list, bypass YOLO and use CNN directly
 6. Filter by confidence threshold
 7. Extract dominant color
 8. Display overlays
 9. Log to memory/export
-

6. Innovation Highlights

- ♦ Multi-model ensemble improves ambiguous predictions
 - ♦ Color tagging provides both function and form
 - ♦ Fallback system ensures robustness when YOLO fails
 - ♦ Class-skipping logic avoids common detection errors
 - ♦ Compatible with webcams and phone cameras
 - ♦ Modular: new parts, models, and outputs can be integrated
-

7. Use Cases

A. Educational Tools

- ♦ Physical-symbol interaction for learning

- Real-time feedback enhances engagement

B. Sorting and Recycling

- Automates part classification and binning
- Helps hobbyists, recycling labs, and DIY sorters

C. Inventory and Storage

- Digital cataloging of LEGO collections
- Part and color logging for inventory apps

D. Research and Prototyping

- Practical example of integrated detection-classification pipeline
- Testbed for ensemble logic, vision pipelines

8. Challenges Addressed

Challenge	LEXIS Solution
Small/flat parts missed by detection	YOLO fallback to CNN
Color mismatches	Euclidean color distance matching
Low-confidence predictions	Confidence thresholds + ensemble
Ambiguous shapes	CNN7 model
Real-time limitations	Frame skipping, optimized inference

L.E.X.I.S Workflow Pipeline – Detailed Process Description

1. Frame Capture

Frames are captured in real-time from either:

- A webcam
- A phone camera streaming over local network

Each frame is analyzed independently.

2. Frame Preparation

- ♦ Frames are resized (e.g., 640x640) for uniform input
 - ♦ Two copies are created:
 - One for display
 - One for analysis
-

3. Object Detection with YOLO

- ♦ YOLO runs on every n -th frame (e.g., every 6 frames)
 - ♦ Output includes:
 - Bounding box coordinates
 - Class ID
 - Confidence score
-

4. Detection Filtering

Detections are filtered by:

- ♦ Confidence threshold (to reduce noise)
 - ♦ Minimum and maximum bounding box area (to ignore irrelevant objects)
-

5. Fallback Detection

When YOLO returns no detections:

- ♦ A fallback crop from the frame center is passed to the classifier
 - ♦ Ensures system continues functioning on difficult parts or bad lighting
-

6. Patch Classification

For each detection:

- ♦ Patch is extracted and passed to MobileNetV2

- ♦ If the class is in the ambiguous list, CNN7 is also used
- ♦ Their outputs are blended for better accuracy

If the confidence is low or class is unknown, it is ignored.

7. Color Detection

- ♦ Center pixel color is extracted from patch
 - ♦ Compared against LEGO color database
 - ♦ Best match (by BGR distance) is selected and labeled
-

8. Result Logging and Display

Each result includes:

- ♦ Detected part
- ♦ Confidence
- ♦ Matched color

Visuals (box + label) are drawn on the frame, and logs are updated.

9. Performance Monitoring

The system calculates FPS to track performance using:

- ♦ Time deltas between processed frames
 - ♦ A moving average (e.g., via deque) to smooth fluctuations
-

10. Continuous Operation

Process repeats on each frame:

- ♦ Detection, classification, color analysis, rendering
- ♦ GUI allows:
 - Switching camera sources
 - Classifying saved images
 - Saving session

CNN Training Code for L.E.X.I.S

1. Imports

```
import os
import time
import cv2
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
```

Purpose of Each Library

Library	Purpose
os , time	File and path handling, timestamps
cv2	Image input/output, webcam integration
numpy	Numerical operations
matplotlib , seaborn	Visualization of training performance (loss, accuracy)
sklearn.metrics	Evaluation metrics (report, confusion matrix)
tensorflow	Core deep learning framework

2. Configuration & Model Setup

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping
```

Keras Components Used:

Component	Purpose
ImageDataGenerator	Image augmentation
MobileNetV2	Transfer learning base model
Model , Layers	Build custom head architecture
Adam	Optimizer
Callbacks	Improve training reliability

```

IMAGE_SIZE = (96, 96)
BATCH_SIZE = 32
EPOCHS = 50
DATASET_PATH = r"C:\Users\Jasmi\Desktop\Machine project\Meow\data6"

```

3. Dataset Loading & Augmentation

```

train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    zoom_range=0.2,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.15,
    horizontal_flip=True,
    brightness_range=[0.8, 1.2],
    fill_mode='nearest'
)

```

Key Transformations

- ♦ Normalize pixel values to 0–1
- ♦ Add randomness to improve generalization

```

train_generator = train_datagen.flow_from_directory(
    DATASET_PATH,
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training',
    shuffle=True
)

```

```

)

val_generator = train_datagen.flow_from_directory(
    DATASET_PATH,
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation',
    shuffle=False
)

```

4. Model Architecture (MobileNetV2 + Custom Head)

```

base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=
(96, 96, 3))
base_model.trainable = True

```

- Loads pretrained model (without classifier)
- Enables fine-tuning (not frozen)

```

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
predictions = Dense(train_generator.num_classes, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=predictions)

```

Custom Head Layers

Layer	Purpose
GAP	Reduces spatial dimensions (flattening)
Dropout	Regularization to reduce overfitting
Dense(128)	Learnable fully connected layer
Dense(softmax)	Output: class probabilities

5. Compilation & Callbacks

```
model.compile(optimizer=Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

- ♦ Loss: Categorical crossentropy (multi-class)
- ♦ Metric: Accuracy

```
lr_schedule = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3,
                                verbose=1)
early_stop = EarlyStopping(monitor='val_loss', patience=7,
                           restore_best_weights=True, verbose=1)
```

Why Use Callbacks?

Callback	Benefit
ReduceLROnPlateau	Reduce learning rate if model stagnates
EarlyStopping	Stop training early to prevent overfitting

6. Handling Class Imbalance

```
from sklearn.utils.class_weight import compute_class_weight

class_indices = train_generator.class_indices
class_labels_array = np.array(list(class_indices.values()))
class_weight = compute_class_weight(class_weight='balanced',
                                   classes=class_labels_array, y=train_generator.classes)
class_weight_dict = dict(enumerate(class_weight))
```

Explanation

Step	Role
compute_class_weight	Calculate weights to balance classes
Underrepresented classes	Get higher weight
Overrepresented classes	Get lower weight
Result	Dict used in model training

7. Model Training

```
history = model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=val_generator,
    callbacks=[lr_schedule, early_stop],
    class_weight=class_weight_dict
)
```

Training Features

- Uses both training and validation sets
- Applies callbacks and class balancing
- Saves training history for analysis

8. Save the Model

```
timestamp = time.strftime("%Y%m%d_%H%M%S")
model_filename = f"mobilenetv2_classifier_{timestamp}.h5"
model.save(model_filename)
```

Why Timestamp?

- Ensures unique filenames
- Helps track model versions

To reload:

```
from tensorflow.keras.models import load_model
model = load_model("mobilenetv2_classifier_20250530_234510.h5")
```

9. Plot Training Curves

```
plt.figure(figsize=(12, 5))
...
```

```
plt.show()
```

Visualize training and validation accuracy and loss across epochs.

10. Evaluate the Model

```
val_generator.reset()
y_true = val_generator.classes
y_pred = model.predict(val_generator)
y_pred_classes = np.argmax(y_pred, axis=1)
```

Steps

Step	Description
reset()	Restart generator for consistent evaluation
y_true	Get actual class labels
predict()	Predict probabilities
argmax()	Convert to predicted class labels

```
report = classification_report(y_true, y_pred_classes,
                               target_names=class_labels)
print("Classification Report:\n", report)
```

- ♦ Precision, recall, F1, support for each class
- ♦ Helpful for detailed diagnosis

```
cm = confusion_matrix(y_true, y_pred_classes)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



- ♦ Confusion matrix to visualize misclassifications
-

Summary

This training pipeline:

- ♦ Leverages transfer learning with MobileNetV2
- ♦ Applies augmentation and class balancing
- ♦ Utilizes callbacks to avoid overfitting
- ♦ Saves models with timestamping
- ♦ Provides detailed evaluation (reports + visualizations)

CNN Classifiers for L.E.X.I.S

The LEXIS system uses two dedicated CNN models to classify LEGO bricks after detection by YOLO or during fallback:

- CNN23: A general-purpose classifier using transfer learning with MobileNetV2, trained on 23 classes of real LEGO parts.
 - CNN7: A smaller, specialized classifier trained on synthetic/augmented data for resolving ambiguity among 7 highly confusing part types.
-

1. CNN23 – MobileNetV2 Classifier

Overview

Feature	Value
Model	MobileNetV2 (pretrained on ImageNet)
Architecture	CNN base + GAP + Dropout + Dense(128) + Dense(23)
Input Size	96 × 96 × 3
Dataset	46,000+ real LEGO images (train + validation)
Classes	23 LEGO part types
Training Epochs	50
Optimizer	Adam (lr = 1e-5)
Callbacks	ReduceLROnPlateau, EarlyStopping

Performance Summary

Final Accuracy:

- Train Accuracy: 92.9%
- Validation Accuracy: 92.3%
- Best Validation Accuracy: 92.8%

Final Loss:

- ♦ Train Loss: 0.198
- ♦ Validation Loss: 0.199

Source: CNN23_TL_MBv2_Results.txt

Classification Report (23 Classes)

Metric	Value
Precision	0.93
Recall	0.93
F1-Score	0.93
Support	9200 samples

Most classes performed above 94% F1, except:

- ♦ (2445) 2×12 Plate – F1: 0.80
- ♦ (3460) 1×8 Plate – F1: 0.71
- ♦ (3832) 2×10 Plate – F1: 0.77
- ♦ (4477) 1×10 Plate – F1: 0.74

These classes are also part of the SKIP_YOLO_CLASSES list.

Confusion Matrix

See: CNN23_TLv2_confusion_matrix.png

- ♦ Most confusion occurs among flat plates (1×8, 2×10, 1×10)
 - ♦ Brick-like parts (e.g., 1×2 Brick, 1×1 Brick) are consistently accurate
-

Accuracy & Loss Curves

See: CNN23_TLv2_accuracy.png

- ♦ Validation accuracy surpassed training by epoch 5
- ♦ Loss continuously decreased with no overfitting
- ♦ Model stabilized around epoch 35

2. CNN7 – Ambiguity Resolver Classifier

Overview

Feature	Value
Model	Custom CNN (lightweight architecture)
Input Size	96 × 96
Classes	7 frequently confused LEGO types
Dataset	Synthetic / Augmented images
Use Case	Ensemble with CNN23 for high-ambiguity classes

Performance Summary

- ♦ Train Accuracy: ~70%
- ♦ Validation Accuracy: ~77%
- ♦ Train Loss: ~0.4
- ♦ Validation Loss: ~0.3
- ♦ Epochs: 10

Source: Visuals: CNN7 3rd genrated data.png , CNN accuracy.png

Confusion Matrix

See: CNN7 synthatic data confusion matrix.png

- ♦ Moderate confusion remains, but the model resolves:
 - 1×2 Brick vs 1×2 Jumper
 - 1×8 Plate vs 2×10 Plate
 - Slope vs Plate (improved with color context + ensemble)
-

3. Ensemble Usage in LEXIS

Integration Logic:

```
if class_id in ENSEMBLE_CLASSES:
    result = blend(cnn23_output, cnn7_output)
```

- CNN7 is not always called – only used when:
 - Part is ambiguous
 - Confidence is near threshold
- Final class decision is based on:
 - Weighted average between CNN23 and CNN7 outputs

4. Model Roles in Pipeline

Stage	Model	Responsibility
Detection	YOLO	Locate part in frame
Primary Classification	CNN23	Predict part from patch
Fallback Classification	CNN23 (center crop)	Used if YOLO fails
Ensemble Refinement	CNN7	Helps in visually ambiguous cases

5. Challenges Addressed

Challenge	Solution
Misclassification of similar plates	CNN7 ensemble logic
Overfitting on rare classes	Dropout + class weights
Low color diversity	Color extraction pipeline added
Model confidence too low on edge cases	Combined CNN outputs

6. Future Work

- Expand CNN7 with real images to improve generalization
- Fine-tune CNN23 with augmented data for skipped YOLO classes
- Add CNN-based color embedding layer for color-aware classification
- Integrate attention mechanisms in CNN7 to focus on local geometry
- Evaluate CNN7 on adversarial examples (tilted, rotated, occluded)

YOLO model for L.E.X.I.S

Overview

The YOLO component of the LEXIS system is a custom-trained YOLOv8n object detector designed specifically to locate LEGO bricks in real-time from webcam or IP camera streams. It performs lightweight, fast detection and passes the regions of interest (ROIs) to CNN classifiers for precise identification and color labeling.

1. Model Details

Attribute	Value
Version	YOLOv8n (Ultralytics)
Framework	PyTorch (via Ultralytics or Roboflow API)
Model Path	YOLO11_optimizd_pic_mode.pt (local deployment)
Training Source	Roboflow Custom Dataset
Detection Classes	14 LEGO part categories (IDs 0–13)
Input Resolution	640 × 640
File Size	~5.4 MB (optimized .pt without optimizer)
Inference Speed	~0.7 ms per image (GPU), 6.3 GFLOPs
Deployment	Webcam (OpenCV), optionally via Roboflow API

2. Training Results

Training was performed for 50 epochs on a LEGO dataset of 1756 instances using Roboflow's pipeline.

Metric	Value
mAP@0.5	0.982
mAP@0.5:0.95	0.914
Precision (avg)	~0.964
Recall (avg)	~0.952

Metric	Value
Best Class	Class 13 (F1 = 0.995)
Worst Class	Class 1 (Recall = 0.851)

Source: YOLO11_Roboflow_trained_model_results.txt

3. Inference Pipeline (Standalone)

The model can be deployed in two modes:

A. Roboflow API Deployment

```
from inference import get_model
model = get_model(model_id="ldc-lego-yolo-5wyps/1",
api_key="w5RGd00YzK6pvmC3EoRx")
```

- Uses Roboflow's hosted version of the model
- Automatically applies preprocessing and postprocessing
- Easy to deploy without handling .pt files locally

B. Local Inference with Ultralytics

```
from ultralytics import YOLO
model = YOLO("YOLO11_optimizd_pic_mode.pt")
results = model(frame)[0]
```

- Manual control over thresholds, filters, and integration
 - Used for fast webcam inference with OpenCV + Tkinter GUI
-

4. Filtering and Postprocessing

Key Filtering Conditions:

```
if conf < YOLO_CONFIDENCE: continue
if class_name in SKIP_YOLO_CLASSES: continue
if area < MIN_AREA or area > MAX_AREA: continue
```

- Confidence Threshold: ≥ 0.6
- Frame Skipping: Only detect every 6 frames
- Skip Logic: Certain classes are excluded from YOLO and handled by CNN
- Bounding Box Size Filtering: Removes extremely small or large false positives

Skipped YOLO Classes:

```
SKIP_YOLO_CLASSES = [  
    '(3008) 1×8 Brick',  
    '(3460) 1×8 Plate',  
    '(3832) 2×10 Plate',  
    '(4477) 1×10 Plate'  
]
```

5. Integration with LEXIS

Combined Workflow:

1. Capture frame from webcam or IP camera
 2. Every 6th frame → YOLO detection
 3. Filter detections by confidence and class logic
 4. For each accepted bounding box:
 - Crop the region
 - Pass to MobileNetV2 (CNN23)
 - If in ensemble set → blend with CNN7 output
 5. Extract color from the patch center
 6. Draw overlay with:
 - Bounding box
 - Part name
 - Confidence
 - Color
 7. Store result in `.xlsx` exportable format
-

6. Strengths

Feature	Benefit
Real-time detection	Inference speed supports video streams
Small model size	Optimized for deployment on low-end GPUs
Modular filtering	Fine control over what YOLO handles vs CNN
Roboflow-trained	Clean data, augmentation, efficient convergence
Flexible deployment	API mode and local .pt mode supported

7. Limitations

Limitation	Description
Weak on long bricks	Flat, wide pieces are skipped or poorly detected
Sensitivity to lighting	Shadows and blur affect detection quality
No temporal tracking	Detection is frame-based, not video-sequence aware
Skipped frames may miss fast motion	If an object appears between detection intervals

Model summary

YOLO summary: 181 layers, 2,592,570 parameters, 0 gradients, 6.5 GFLOPs

181 layers

- ♦ This model has 181 layers, including convolutional layers, batch norm, activation functions, detection heads, etc.
- ♦ These layers are stacked to form the full YOLO neural network pipeline.

Interpretation: This is a compact YOLO variant (probably YOLOv8n or YOLOv5n — the "n" means nano: a very lightweight model for speed and low-resource use).

2,592,570 parameters

- ♦ These are the trainable weights and biases in the model.
- ♦ They're what the model learned during training (like features of LEGO parts).

For comparison:

- ♦ YOLOv8n ~3M parameters
- ♦ YOLOv8s ~11M
- ♦ YOLOv8m ~25M

Interpretation: Your model is tiny, optimized for fast inference and low power devices (good for Raspberry Pi, Jetson Nano, etc.).

0 gradients

- ♦ Gradients are only populated during training — they're used for updating parameters via backpropagation.
 - ♦ Since you're doing inference (not training), gradients are zero.
-

6.5 GFLOPs

- ♦ GFLOPs = Giga Floating Point Operations = how much computation is needed to process one image.
- ♦ 6.5 GFLOPs means it does 6.5 billion operations per forward pass.

Interpretation: That's relatively lightweight. For example:

- ♦ MobileNetV2 ~300–600 MFLOPs
- ♦ ResNet50 ~4 GFLOPs
- ♦ YOLOv8m ~30+ GFLOPs

This means your model can run fast on most machines, even in real time.

TL;DR (Summary)

Item	Meaning
181 layers	Network depth (tiny YOLO version)

Item	Meaning
2.59M params	Small model — fast and light
0 gradients	You're in inference mode
6.5 GFLOPs	Lightweight — good for real-time performance

Architecture Breakdown

1. Backbone (Feature Extractor)

This is where the image goes in, and features are extracted using convolutional layers.

- Focus Layer (optional in v8): Converts image to feature map.
- C2f Modules: Efficient convolutional blocks with shortcut connections (like residuals).
- Downsampling via stride-2 convolutions (reduces resolution but increases depth).

2. Neck (Feature Aggregator)

Combines features from multiple scales.

- FPN (Feature Pyramid Network): Combines high-res and low-res features.
- PAN (Path Aggregation Network): Enhances object localization and recognition.

3. Head (Detection Layer)

Predicts bounding boxes and classes.

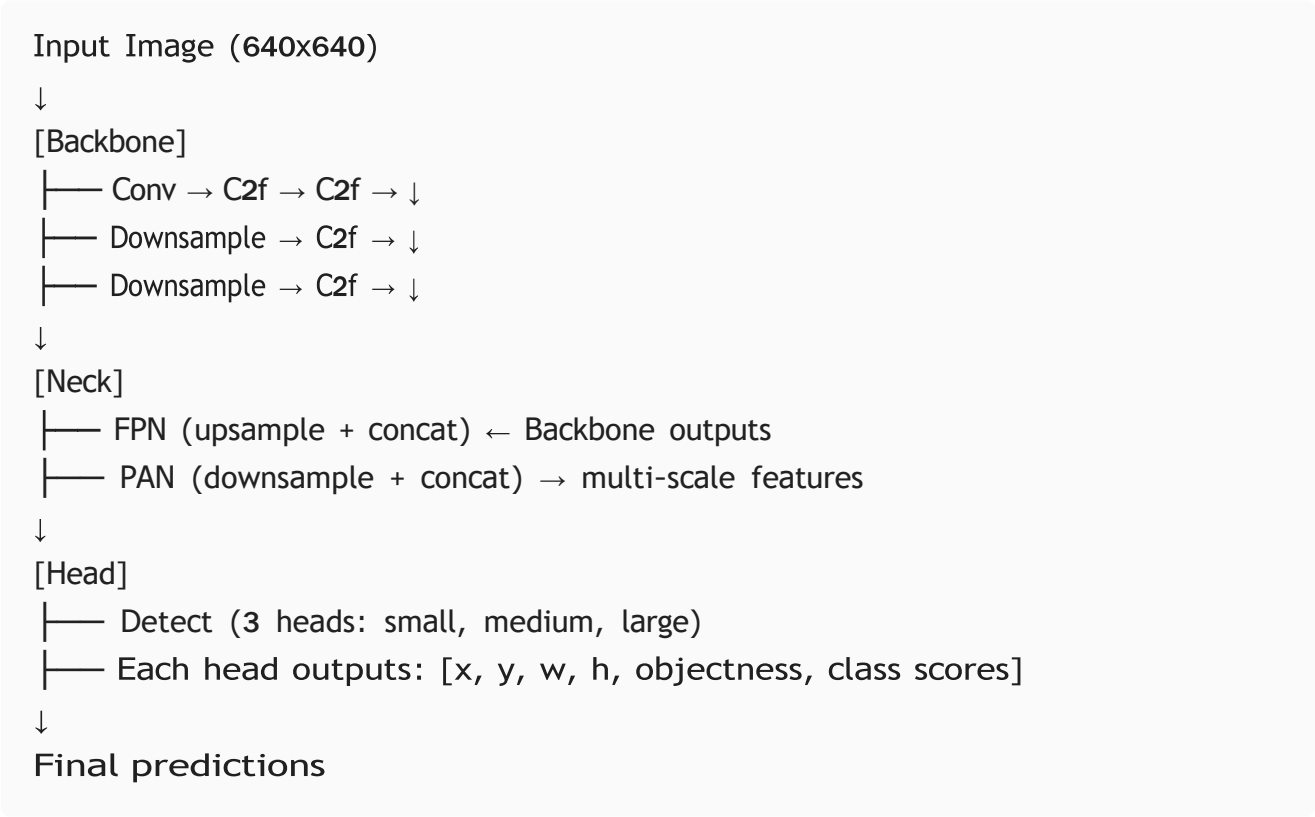
- Uses anchor-free detection (unlike YOLOv5).
- Outputs:
 - (x, y, w, h) → box coordinates
 - objectness score
 - class scores (softmax or sigmoid)

Layer Types & Counts in YOLOv8n (Approximate)

Section	Layer Types	Example Count
Backbone	Conv, BatchNorm, SiLU, C2f blocks	~70 layers

Neck	Upsample, Concat, C2f, PAN, FPN	~60 layers
Section	Layer Types	Example Count
Head	Conv, Detect, Output layers	~50 layers
Total	All combined	181 layers

Visual Diagram (Text Representation)



YOLO-CNN Hybrid method Code for L.E.X.I.S

1. Imports & Setup

```
import cv2, numpy as np, time, threading, tkinter as tk
from tkinter import filedialog, simpledialog
import pandas as pd
from collections import deque
from tensorflow.keras.models import load_model
from ultralytics import YOLO
from PIL import Image, ImageTk
```

- ♦ These imports cover:
 - ♦ **cv2** (OpenCV): camera access, frame manipulation.
 - ♦ **numpy** : numerical array handling.
 - ♦ **time** : time tracking for FPS.
 - ♦ **threading** : run camera streams in background.
 - ♦ **tkinter** : GUI with buttons and image display.
 - ♦ **pandas** : storing/exporting results.
 - ♦ **deque**: efficient FPS history.
 - ♦ **keras** : loading trained CNN models.
 - ♦ **ultralytics** : loading YOLO model.
 - ♦ **PIL** : convert OpenCV images to displayable format.

2. Configuration Parameters

```
IMAGE_SIZE = (96, 96)
FRAME_RESOLUTION = (640, 640)
CONFIDENCE_THRESHOLD = 0.4
YOLO_CONFIDENCE = 0.8
ENSEMBLE_ALPHA = 0.5
YOLO_SKIP_FRAMES = 6
```

- ♦ Image Size: for CNN input (96x96).
- ♦ Resolution: webcam frame resolution.

- Thresholds: for classifying as valid.
- Ensemble Alpha: weight between CNN23 and CNN7.
- Skip Frames: YOLO doesn't run every frame to save processing.

Configuration Parameters (Line-by-Line Breakdown)

```
IMAGE_SIZE = (96, 96)
```

- This defines the input size for the CNN models (MobileNetV2 and CNN7). All
 - images passed to the CNNs will be resized to 96×96 pixels.
 - Why?
 - CNNs expect fixed input dimensions.
 - Smaller sizes reduce computational load.
 - 96×96 is the least allowed resolution for MobileNetV2.
-

```
FRAME_RESOLUTION = (640, 640)
```

- Sets the resolution of video frames shown in the display and processed.
 - Applies to both webcam and phone stream inputs.
 - A square resolution (640x640) is used because:
 - YOLOv8 performs better with square images (trained that way).
 - Keeps object bounding box dimensions consistent.
 - Larger resolution means:
 - More pixels to analyze → better detail.
 - But slower processing → trade-off between speed and accuracy.
-

```
CONFIDENCE_THRESHOLD = 0.4
```

- This is the minimum confidence score required for a classification result to be accepted from the CNNs.
- If a CNN (MobileNet or ensemble) predicts a class with less than 40% certainty, the result is discarded and labeled as "Unknown".
- Purpose:
 - Avoid showing uncertain predictions.
 - Keep the system robust and trustworthy.

YOLO_CONFIDENCE = 0.8

- ♦ Minimum confidence level required for a YOLO-CNN object detection result to be accepted.
- ♦ Detections below 80% confidence are ignored.
- ♦ This is intentionally set high, because:
 - YOLO sometimes detects irrelevant or partial objects in clutter.
 - LEGO parts are often small, and overlapping → high confidence filters out bad boxes.
- ♦ Setting it too low would show many false positives.

ENSEMBLE_ALPHA = 0.5

- ♦ Used when combining predictions from CNN23 and CNN7 in an ensemble fashion.
- ♦ This is a weighted average:

$$\text{final_prediction} = \text{ALPHA} * \text{CNN23_output} + (1 - \text{ALPHA}) * \text{CNN7_output}$$

- ♦ 0.5 means equal weight to both models.
- ♦ Purpose:
 - Some classes are better predicted with CNN7 due to synthetic data specialization.
 - This blending can improve accuracy for overlapping/confusing parts.

YOLO_SKIP_FRAMES = 6

- ♦ YOLO object detection is not run on every frame. Instead,
- ♦ it is run once every 6 frames to:
 - Save processing time.
 - Improve performance on lower-end machines or phone camera feeds.
- ♦ Why this works:
 - LEGO objects don't move rapidly between frames.
 - Detected bounding boxes from previous frames are reused in skipped frames (via caching).

Summary of Intent

This configuration block balances:

- Accuracy (via high confidence thresholds and ensemble logic),
- Performance (by skipping frames and using smaller input sizes),
- Flexibility (square frame resolution works for both detection and classification),
- and Efficiency (smart fallback between YOLO and CNN based on class type or detection failure).

3. Model Paths & Loading

```
MOBILENET_MODEL_PATH = r"...CNN_LEGO_weights_23Classes.h5"  
YOLO_MODEL_PATH = r"...YOLO11_optimizd_pic_mode.pt"  
MODEL_7CLASS_PATH = r"...CNN_Synthatic_Data_7Classes.h5"
```

- Paths to pretrained models.

```
yolo_model = YOLO(YOLO_MODEL_PATH)  
mobilenet_model = load_model(MOBILENET_MODEL_PATH)  
cnn7_model = load_model(MODEL_7CLASS_PATH)
```

- Load models into memory.

Model Path Variables

These variables are strings containing absolute paths to the files where your trained models are stored.

```
MOBILENET_MODEL_PATH = r"...CNN_LEGO_weights_23Classes.h5"
```

- This path points to a `.h5` file, which is a Keras model saved in HDF5 format.
- The model is a MobileNetV2-based CNN trained to classify 23 different LEGO part classes.

- ♦ It is referred to as CNN23 in your system.
 - ♦ Used for: main classification for most parts.
-

```
YOLO_MODEL_PATH = r"...YOLO11_optimizd_pic_mode.pt"
```

- ♦ This is a path to a `.pt` file: a PyTorch format model, specifically for YOLOv8 (from the `ultralytics` package).
 - ♦ YOLO (You Only Look Once) is an object detection model.
 - ♦ This particular model was trained to detect LEGO parts in images.
 - ♦ Used for: detecting where the LEGO pieces are in the video frame, i.e., drawing bounding boxes.
-

```
MODEL_7CLASS_PATH = r"...CNN_Synthatic_Data_7Classes.h5"
```

- ♦ Another `.h5` file, but this one contains a CNN model trained on synthetic data (e.g., augmented, digitally rendered).
 - ♦ It classifies 7 specific LEGO classes with higher accuracy.
 - ♦ Used in ensemble mode: the system combines its output with MobileNet when the class belongs to a "special set".
-

Model Loading

These lines actually load the models from disk into memory so they can be used in inference:

```
yolo_model = YOLO(YOLO_MODEL_PATH)
```

- ♦ Loads the YOLOv8 model using the Ultralytics API.
 - ♦ After loading, `yolo_model.predict(...)` can be used on frames to detect objects (bounding boxes, scores, and class IDs).
 - ♦ YOLO detects all LEGO bricks as *objects*, not their class type (that comes later from CNN).
-

```
mobilenet_model = load_model(MOBILENET_MODEL_PATH)
```

- ♦ Loads the 23-class CNN classifier into memory. is
 - ♦ `load_model` a Keras function from TensorFlow.
 - ♦ After loading, you can call `mobilenet_model.predict(...)` on an image to get class probabilities.
-

```
cnn7_model = load_model(MODEL_7CLASS_PATH)
```

- ♦ Loads the 7-class CNN into memory.
 - ♦ This model is only used when the detected class belongs to a predefined set (`ENSEMBLE_CLASSES`), because those classes benefit from extra refinement.
 - ♦ In those cases, the results from this model are combined with MobileNet's using a weighted average.
-

Why Use 3 Models?

Model	Purpose	Framework	Input Type
YOLO	Detect bounding boxes (where)	PyTorch	Full video frame
mobilenet_model	Classify detected patch (what)	Keras / TF	Cropped object image
cnn7_model	Refine classification (7 classes)	Keras / TF	Same cropped image

Using multiple models lets you:

- ♦ Combine fast object detection (YOLO) with accurate classification (CNNs).
 - ♦ Fallback to MobileNet when YOLO is unsure or bad at detecting certain parts. Use
 - ♦ ensemble logic to boost precision on ambiguous parts.
-

4. Labels & Classification Lists

```

CLASS_LABELS = [ ... ] # All 23 classes
ENSEMBLE_CLASSES = [ ... ] # Classes to be refined with CNN7
SKIP_YOLO_CLASSES = ['(3008)...', '(3460)...'] # Only use MobileNet for these

```

- `CLASS_LABELS` : all classes trained in CNN23.
- `ENSEMBLE_CLASSES` : subset that uses both CNN23 + CNN7.
- `SKIP_YOLO_CLASSES` : classes that YOLO is bad at, so only CNN is trusted.

CLASS_LABELS`

```

CLASS_LABELS = [
    "(11214) Technic Axle Pin 3L with Friction Ridges Lengthwise and 1L Axle",
    "(15068) 2×2 Curved",
    ...
    "(98138) 1×1 Tile, Round"
]

```

- This is the master list of 23 LEGO classes your MobileNetV2 model (CNN23) is trained on.
- Each entry corresponds to a class index (0 to 22) in the model's output.
- When you call:

```

preds = mobilenet_model.predict(img)
class_id = np.argmax(preds)
label = CLASS_LABELS[class_id]

```

It maps the highest probability index to its human-readable label.

- These names are the official part names from LEGO part catalogs, with IDs like (3004) , (3460) , etc.

Usage:

- This list is the reference for assigning final predictions to actual LEGO part names.

ENSEMBLE_CLASSES`

```

ENSEMBLE_CLASSES = [
    "(3004) 1×2 Brick", "(3005) 1×1 Brick", "(3022) 2×2 Plate",

```

```
"(3623) 1×3 Plate", "(4286) Slope 33 3 x 1",
"(11214) Technic Axle Pin 3L with Friction Ridges Lengthwise and 1L
Axle",
"(15573) 1×2 Jumper"
]
```

- This is a subset of `CLASS_LABELS` that are known to be confusing or ambiguous.
- These specific classes benefit from ensemble classification, where two models are combined:
 1. CNN23 (MobileNet)
 2. CNN7 – a specialized model trained on synthetic or more controlled data.
- When a prediction falls into this list, the code blends the outputs:

```
ensemble_preds =  $\alpha$  * CNN23 + (1 -  $\alpha$ ) * CNN7
```

- This results in a smarter, more confident prediction.
- These classes tend to be geometrically similar, e.g., different brick sizes or parts with small visual differences.

Usage:

- Improves classification precision.
- You use CNN7 only when you know it will help—not every time, which saves computation.

SKIP_YOLO_CLASSES

```
SKIP_YOLO_CLASSES = [
    '(3008) 1×8 Brick', '(3460) 1×8 Plate',
    '(3832) 2×10 Plate', '(4477) 1×10 Plate'
]
```

- These are classes that YOLO (the object detector) is bad at detecting reliably.
- Typically because:
 - They are long and flat, making bounding boxes inaccurate.
 - They often get missed or misaligned in detection due to their shape or similarity to background.
- So, instead of using YOLO to detect these, you disable detection for them:
 - If YOLO detects one of these → ignore.

- If nothing is detected in the frame → fallback to center crop and classify with MobileNet only.
- ♦ These parts are always handled using CNN classification, not bounding boxes.

Usage:

- ♦ Improves reliability.
 - ♦ Forces the system to rely on CNN where YOLO is not trustworthy.
-

How These Lists Work Together

Here's a flow of what happens:

1. YOLO runs and finds bounding boxes.
2. Each patch from a box is sent to `classify_patch()`.
3. Inside `classify_patch()`:
 - ♦ The CNN23 model is used.
 - ♦ If the predicted class is in `ENSEMBLE_CLASSES`:
 - CNN7 is also used and blended in.
 - ♦ If the predicted class is in `SKIP_YOLO_CLASSES`:
 - Ignore the YOLO box and just use the CNN result.
4. The result is only shown if confidence > `CONFIDENCE_THRESHOLD`.
5. The result is shown as YOLO would normally show on boxes unless its not detecting any box

5. Color Matching Function

```
def find_closest_lego_color(bgr_tuple):
```

- ♦ Maps average color of the patch to the closest LEGO color based on hardcoded RGB hex values.
- ♦ Calculates Euclidean distance in color space.

This function is designed to:

- ♦ Take the color of a LEGO brick (as a BGR pixel value),
- ♦ And return the name of the closest LEGO color (like "Bright Red", "Sand Blue", etc.)

- ♦ This adds semantic color meaning to classified LEGO parts for logging, sorting, or display.
-

Full Function Signature

```
def find_closest_lego_color(bgr_tuple):
```

- ♦ Input: A BGR tuple like (blue, green, red) , typically sampled from the center pixel of a detected object.
 - ♦ Output: A string like "Bright Red" or "Sand Green" —the name of the closest predefined LEGO color.
-

Step 1: LEGO Color Palette (in HEX)

```
lego_color_hex = {  
    'White': 0xffffff,  
    'Brick Yellow': 0xD9BB7B,  
    'Nougat': 0xD67240,  
    'Bright Red': 0xff0000,  
    'Bright Blue': 0x0000ff,  
    ...  
    'Sand Green': 0x5F8265,  
}
```

- ♦ This dictionary is a hardcoded mapping of official LEGO color names to their RGB color values in hexadecimal format.
 - ♦ Example: 0xff0000 = (255, 0, 0) = Bright Red.
 - ♦ These color codes were chosen based on LEGO's standard palette, and are real-world approximations of what each brick color looks like.
-

Step 2: HEX → BGR Conversion

```
def hex_to_bgr(hex_val):  
    return (hex_val & 0xFF, (hex_val >> 8) & 0xFF, (hex_val >> 16) & 0xFF)
```

- ♦ This function converts the hexadecimal RGB color into a tuple formatted as BGR (Blue, Green, Red).
 - ♦ Why BGR?
 - OpenCV uses BGR by default, not RGB.
 - ♦ Example:
 - `0x478CC6 = RGB(71, 140, 198) → returns (198, 140, 71) in BGR.`
-

Step 3: Color Distance Function

```
def color_distance(c1, c2):  
    return sum((a - b) ** 2 for a, b in zip(c1, c2))
```

- ♦ Calculates the Euclidean distance between two color vectors in BGR space: $\text{distance} = (B_1 - B_2)^2 + (G_1 - G_2)^2 + (R_1 - R_2)^2$
 - ♦ This tells us how visually different two colors are.
-

Step 4: Find the Closest Match

```
return min(lego_color_hex, key=lambda name:  
color_distance(hex_to_bgr(lego_color_hex[name]), bgr_tuple))
```

- ♦ For every LEGO color in the palette:
 1. Convert the color's hex value to BGR.
 2. Compare it with the input color (from the brick image).
 3. Compute the color distance.
 - ♦ `min(..., key=...)` picks the color with the smallest distance, i.e., most visually similar color.
-

Example Run

```
bgr_tuple = (20, 100, 240) # Some shade of reddish-orange  
find_closest_lego_color(bgr_tuple)  
→ "Bright Red"
```

This would return "Bright Red" if that color had the shortest Euclidean distance in BGR space.

Summary

Component	Purpose
lego_color_hex	Reference LEGO colors with true color values
hex_to_bgr()	Convert from standard hex color to OpenCV-compatible BGR
color_distance()	Quantify color difference mathematically
min(..., key=...)	Find the closest color based on visual similarity

Why Is This Useful?

- LEGO bricks are defined not just by shape, but also by color. Including color in classification:
 - Helps with storage and sorting systems.
 - Allows color-based filtering (e.g., "find all red bricks").
 - Enhances UX when visualizing results.
 - Very important for tiny pieces where shape alone may not be enough to classify accurately.
-

6. Image Pre-processing

```
def preprocess_image(img):
```

- Resize to 96x96.
- Normalize to [0, 1].
- Expand dims to match CNN input shape (1, 96, 96, 3) .

Function Definition

```
def preprocess_image(img):
```

This function takes a raw image (or cropped patch from a larger image) and prepares it for input into a convolutional neural network (CNN), specifically MobileNetV2 and CNN7, which require standardized input formats.

Step-by-Step Breakdown

1. Resize the Image

```
img = cv2.resize(img, IMAGE_SIZE)
```

- The image is resized to (96, 96) pixels using OpenCV.
 - IMAGE_SIZE is predefined as (96, 96) in the configuration section.
 - Why?
 - CNN models require fixed-size input tensors.
 - The model was trained on 96x96 images, so all input images must be reshaped to match.
 - A mismatch in dimensions would result in an error during inference.
-

2. Normalize Pixel Values

```
img = img.astype("float32") / 255.0
```

- The image's pixel values, originally in the range [0, 255], are converted to floating-point numbers and scaled down to the range [0.0, 1.0].
 - This is a standard practice in deep learning because:
 - It improves convergence during training.
 - Helps the model generalize better.
 - Prevents issues caused by large gradients.
 - This line ensures that the input matches the preprocessing that was used during training.
-

3. Expand Dimensions

```
return np.expand_dims(img, axis=0)
```

- Converts the shape of the image from `(96, 96, 3)` to `(1, 96, 96, 3)`.
 - This adds a batch dimension of size 1.
 - CNNs expect a batch of images as input, even if you're only classifying one image at a time.
 - Final shape:
 - Before: `Height x Width x Channels = 96 x 96 x 3`
 - After: `Batch x Height x Width x Channels = 1 x 96 x 96 x 3`
-

Summary of Purpose

The `preprocess_image()` function performs three key operations:

Operation	Purpose
Resize	Match CNN's expected input dimensions
Normalize	Scale pixel values for numerical stability
Expand dims	Convert to batch format for CNN input requirements

Practical Example

Suppose you crop a LEGO brick patch from a webcam frame. Its original size might be `(120, 160, 3)`. After calling `preprocess_image(patch)`, the output will be:

- Resized to: `(96, 96, 3)`
- Scaled to: `float32` values in `[0.0, 1.0]`
- Expanded to: `(1, 96, 96, 3)`

This makes the image ready for prediction using:

```
predictions = mobilenet_model.predict(preprocessed_image)
```

7. Patch Classification

```
def classify_patch(img):
```

- Skips empty/blank inputs.
- Runs CNN23 model, gets class with highest confidence.
- If the class is in `ENSEMBLE_CLASSES`, run CNN7, and combine predictions with weighted average using `ENSEMBLE_ALPHA`.
- If confidence < threshold, returns "Unknown".

Function Definition

```
def classify_patch(img):
```

This function is responsible for classifying a single image patch (typically cropped from a bounding box or the center of the frame) into one of the predefined LEGO part classes using your trained CNN models.

Step-by-Step Breakdown:

Step 1: Handle Invalid or Empty Inputs

```
if img is None or img.size == 0 or np.mean(img) < 10 or np.std(img) < 20:  
    return "Unknown", 0.0
```

- This line filters out poor-quality inputs:
 - `img is None`: input is not valid.
 - `img.size == 0`: image is empty or has no pixels.
 - `np.mean(img) < 10`: image is nearly black (too dark to recognize).
 - `np.std(img) < 20`: image lacks contrast (possibly blurred or overexposed).
 - If any of these conditions are true:
 - Return "Unknown" with confidence 0.0 to skip classification.
-

Step 2: Preprocess the Image

```
preprocessed = preprocess_image(img)
```

- Prepares the image using:
 - Resize to (96, 96)
 - Normalize pixel values to [0, 1]

- Add a batch dimension for the CNN
 - ♦ Result: a tensor of shape (1, 96, 96, 3) ready for inference.
-

Step 3: Predict Using CNN23 (MobileNetV2)

```
preds_23 = mobilenet_model.predict(preprocessed, verbose=0)[0]
class_id_23 = np.argmax(preds_23)
label_23 = CLASS_LABELS[class_id_23]
conf_23 = preds_23[class_id_23]
```

- ♦ Pass the preprocessed image into the CNN23 model (MobileNetV2).
 - ♦ `preds_23` is an array of probabilities for each class.
 - ♦ `np.argmax(preds_23)`: finds the index of the class with the highest confidence.
 - ♦ `label_23`: looks up the corresponding label from `CLASS_LABELS`.
 - ♦ `conf_23`: retrieves the highest confidence value.
-

Step 4: If Class is in `ENSEMBLE_CLASSES`, Refine with CNN7

```
if label_23 in ENSEMBLE_CLASSES:
    preds_7 = cnn7_model.predict(preprocessed, verbose=0)[0]
    preds_7_padded = np.zeros_like(preds_23)
    preds_7_padded[:7] = preds_7
    ensemble_preds = ENSEMBLE_ALPHA * preds_23 + (1 - ENSEMBLE_ALPHA) *
preds_7_padded
    class_id = np.argmax(ensemble_preds)
    confidence = ensemble_preds[class_id]
    label = CLASS_LABELS[class_id] if confidence >= CONFIDENCE_THRESHOLD
else "Unknown"
    return label, confidence
```

- ♦ For certain LEGO classes that are difficult to classify, an ensemble of two models is used:
 1. CNN23 (general-purpose classifier).
 2. CNN7 (specialized on 7 specific LEGO parts).`preds_7` contains the output of the CNN7 model.
- ♦ Since `preds_7` only contains 7 classes, it is padded to match the size of `preds_23`
- ♦ using `np.zeros_like`.

- This ensures alignment in shape before combining.
 - ♦ `ensemble_preds` blends the outputs: $\text{ensemble} = \alpha \cdot \text{CNN23} + (1 - \alpha) \cdot \text{CNN7}$
 - ♦ The label is returned only if the confidence exceeds the threshold. Otherwise, is "Unknown" returned.
-

Step 5: For Other Classes, Use CNN23 Alone

```
else:  
    label = label_23 if conf_23 >= CONFIDENCE_THRESHOLD else "Unknown"  
    return label, conf_23
```

- ♦ If the predicted class is not in the ensemble list:
 - Only the CNN23 prediction is used.
 - If the confidence is below `CONFIDENCE_THRESHOLD` (0.4), label is set to "Unknown".
-

Purpose Summary

Step	Purpose
1	Rejects low-quality images (noise, darkness, blur)
2	Prepares input to match CNN format
3	Gets base classification from CNN23 (MobileNetV2)
4	If class is hard to classify, blend with CNN7 to improve accuracy
5	Return best guess only if confident; otherwise mark as unknown

Practical Implications

- ♦ Ensures that only valid and confident predictions are accepted. Uses
- ♦ additional model support (ensemble) when needed.
- ♦ Helps avoid misclassifications that can arise from:
 - Overlapping bricks
 - Partial visibility
 - Similar shapes

This function plays a central role in bridging object detection and semantic classification in your LEGO sorting pipeline.

8. Main Video Capture Logic

```
def run_video_capture(source=0):
```

This function is the core loop of the real-time system. It handles:

1. Opening the video stream (webcam or phone IP cam).
 2. Performing frame-by-frame analysis.
 3. Running object detection (YOLO).
 4. Running image classification (MobileNetV2 + CNN7).
 5. Displaying results with bounding boxes, labels, confidence, and color.
 6. Managing performance using frame skipping.
-

Step-by-Step Breakdown

Step 1: Open Video Stream

```
cap = cv2.VideoCapture(source)
if not cap.isOpened():
    print("Failed to open video source.")
    return
```

- ◆ Initializes a video stream using OpenCV.
 - ◆ `source=0` refers to the default webcam, but this can be replaced with a URL (e.g., from a phone camera).
 - ◆ If the stream fails to open, the function exits.
-

Step 2: Setup Frame Tracking and History

```
fps_history = deque(maxlen=10)
frame_count = 0
prev_filtered_boxes = []
```

- ♦ `fps_history` : Stores the most recent 10 FPS values to compute a smoothed average.
 - ♦ `frame_count`: Counts how many frames have been processed.
 - ♦ `prev_filtered_boxes` : Caches YOLO-detected boxes so they can be reused during skipped frames.
-

Step 3: Main Processing Loop

```
while True:
```

This loop runs until the user manually stops the system.

Step 4: Read Frame and Prepare Display

```
ret, frame = cap.read()
if not ret:
    break

display_frame = cv2.resize(frame.copy(), FRAME_RESOLUTION)
```

- ♦ Reads a frame from the video source.
 - ♦ If reading fails (e.g., stream ends), the loop breaks.
 - ♦ Makes a copy of the frame, resized for consistent display dimensions (640x640).
-

Step 5: YOLO Detection on Skipped Frames

```
if frame_count % YOLO_SKIP_FRAMES == 0 or not prev_filtered_boxes:
    results = yolo_model.predict(source=frame, conf=YOLO_CONFIDENCE,
    verbose=False)
    ...
    prev_filtered_boxes = filtered_boxes
```

```
else:
    filtered_boxes = prev_filtered_boxes
```

- YOLO is run only once every `YOLO_SKIP_FRAMES` frames (e.g., every 6th frame).
- If this is a skipped frame, it reuses bounding boxes from the previous detection.
- This saves computational time and allows real-time performance.

Step 6: YOLO Output Filtering

```
for box in boxes:
    x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
    score = float(box.conf[0])
    class_id = int(box.cls[0])
    area = (x2 - x1) * (y2 - y1)
    if score < YOLO_CONFIDENCE or not (MIN_AREA <= area <= MAX_AREA):
        continue
    filtered_boxes.append({...})
```

- For each detected box:
 - Extracts coordinates, confidence score, class ID.
 - Calculates bounding box area.
- Only keeps boxes that:
 - Have a high confidence score (`score >= YOLO_CONFIDENCE`).
 - Have a reasonable size (`MIN_AREA <= area <= MAX_AREA`).

This avoids small noise detections and false positives.

Step 7: Fallback When YOLO Fails

```
if not filtered_boxes:
    h, w = frame.shape[:2]
    center_crop = frame[h//4:3*h//4, w//4:3*w//4]
    ...
    label, conf = classify_patch(center_crop)
    ...
```

- If YOLO detects nothing:
 - A square crop is taken from the center of the frame (50% of the width and height).

- This area is classified using `classify_patch()`.

Why?

- When YOLO fails (e.g., blurry image, thin bricks), this fallback gives the CNN a chance to still classify something.

Step 8: If YOLO Boxes Exist, Process Them

```
for box_info in filtered_boxes:
    ...
    patch = frame[y1:y2, x1:x2]
    label, conf = classify_patch(patch)
    ...
```

- For each YOLO-detected region:
 - A small padding is added to make the box slightly larger.
 - The region is cropped and passed to the classifier.
 - If the result is in `SKIP_YOLO_CLASSES` or is "Unknown", the classifier is run again to double-check.

Step 9: Color Extraction and Result Logging

```
b, g, r = patch[...]
lego_color = find_closest_lego_color((b, g, r))
results_list.append({"Label": label, "Confidence": conf, "Color":
lego_color})
```

- The color of the object is extracted from the center pixel of the patch.
- `find_closest_lego_color()` maps it to the closest LEGO color name.
- The result is logged in `results_list` for future export.

Step 10: Draw Bounding Boxes and Labels

```
cv2.rectangle(display_frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2.putText(display_frame, text, (x1, y1 - 10), ...)
```

- ♦ The detection is visualized:
 - Green rectangle shows the bounding box.
 - Text shows label, confidence, and color.
-

Step 11: FPS Display and Frame Control

```
current_fps = 1.0 / (fps_end_time - fps_start_time + 1e-5)
fps_history.append(current_fps)
...
cv2.putText(display_frame, f"FPS: {smoothed_fps:.1f}", ...)
```

- ♦ Measures how long each frame took to process.
 - ♦ Displays smoothed average FPS on screen.
-

Step 12: Show the Frame and Handle Exit

```
cv2.imshow("LEGO Detector", display_frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```

- ♦ Shows the annotated frame in a window.
 - ♦ If the user presses `q`, the loop breaks and the program ends.
-

Step 13: Cleanup

```
cap.release()
cv2.destroyAllWindows()
```

- ♦ Releases the camera resource.
 - ♦ Closes all OpenCV windows.
-

Summary of Function Responsibilities

Task	Description
Video input	Starts webcam or phone camera stream
Object detection (YOLO)	Identifies regions containing LEGO parts
Frame skipping	Reduces detection frequency to improve real-time performance
Classification (CNN)	Classifies each cropped part using CNN23 or CNN23 + CNN7
Color detection	Identifies the closest LEGO color based on pixel values
Result logging	Stores label, confidence, and color for export
Display	Draws bounding boxes, labels, confidence, and FPS on screen
Fallback logic	Center-classifies if YOLO fails

Patch Classification Inside the Loop

This section occurs after YOLO detects bounding boxes within a frame. Each bounding box is processed to classify the object it surrounds.

Step-by-Step Explanation

1. Padding the Bounding Box

```
pad_x = int((x2 - x1) * 0.1)
pad_y = int((y2 - y1) * 0.1)
x1, y1 = max(0, x1 - pad_x), max(0, y1 - pad_y)
x2, y2 = min(frame.shape[1], x2 + pad_x), min(frame.shape[0], y2 + pad_y)
```

- ♦ The bounding box is expanded by 10% on each side.
 - ♦ Purpose:
 - To ensure the entire brick is included.
 - Avoid clipping edges caused by tightly fitted YOLO boxes.
-

2. Cropping the Patch

```
patch = frame[y1:y2, x1:x2]
```

- ♦ Extracts the image region inside the padded bounding box.
 - ♦ This patch is passed to the classifier.
-

3. Initial Classification

```
label, conf = classify_patch(patch)
```

- ♦ Classifies the cropped patch using your `classify_patch()` function.
 - ♦ This may internally use MobileNetV2 or the ensemble of CNN23 + CNN7.
-

4. Reclassification If Needed

```
if label in SKIP_YOLO_CLASSES or label == "Unknown":  
    label, conf = classify_patch(patch)
```

- ♦ If YOLO is known to be bad at this class (as listed in `SKIP_YOLO_CLASSES`), or the model is unsure (`"Unknown"`), classification is re-run.
 - ♦ This gives the CNN a second chance to analyze the image patch independently of YOLO.
-

5. Store Only Confident Results

```
if label == "Unknown" or conf < 0.25:  
    continue
```

- ♦ If the label is still `"Unknown"` or the confidence is below 25%, the patch is ignored.
 - ♦ This prevents the system from logging or displaying uncertain or incorrect classifications.
-

6. Record and Display the Result

```
results_list.append({...})  
cv2.rectangle(...)
```

```
cv2.putText(...)
```

- The label, confidence, and detected color are stored in `results_list`.
 - The bounding box is drawn on the frame, and the label is shown with:
 - Name of the part
 - Confidence percentage
 - Detected LEGO color
 - YOLO confidence if available
-

FPS Estimation (Frame Rate Smoothing)

Purpose

Real-time systems often have variable frame rates due to processing load. A smoothed FPS value gives the user a better sense of average performance.

Implementation

```
fps_history = deque(maxlen=10)
...
fps_start_time = time.time()
...
fps_end_time = time.time()
current_fps = 1.0 / (fps_end_time - fps_start_time + 1e-5)
fps_history.append(current_fps)
smoothed_fps = sum(fps_history) / len(fps_history)
```

- `deque(maxlen=10)` stores the last 10 frame times.
- `current_fps` is calculated for the current frame using the time it took to process.
- The average of the last 10 FPS values is computed to get `smoothed_fps`.
- This value is displayed on the screen using:

```
cv2.putText(display_frame, f"FPS: {smoothed_fps:.1f}", ...)
```

Benefits

- Prevents large jumps or flickering in the FPS display.
 - Gives a more accurate reflection of system performance.
-

9. Camera and File Classifiers

```
def run_webcam():
    threading.Thread(target=run_video_capture, args=(0,)).start()

def run_phone_cam():
    ip = simpledialog.askstring(...)
    threading.Thread(target=run_video_capture, args=(ip,)).start()
```

- ♦ Start video capture for webcam or phone IP stream in a new thread.

These two functions allow the user to choose the input source for the video stream — either the default webcam or a phone camera via IP stream.

Both functions rely on a common backend function: `run_video_capture(source)`, where `source` is either:

- ♦ `0` → system webcam
- ♦ A URL → phone camera stream (e.g., `http://192.168.x.x:8080/video` from an IP Webcam app)

Function 1: Webcam Mode

```
def run_webcam():
    threading.Thread(target=run_video_capture, args=(0,)).start()
```

Explanation

- ♦ This function launches `run_video_capture(0)` in a separate thread.
- ♦ `0` is the default device index used by OpenCV to refer to the system's first webcam.
- ♦ The use of `threading.Thread(...).start()` :
 - ♦ Ensures that the video stream runs asynchronously, meaning the GUI remains responsive.
 - ♦ Prevents the `run_video_capture()` loop from blocking the main application (i.e., the GUI).

Function 2: Phone Camera Mode

```
def run_phone_cam():
    ip = simpdialog.askstring("Phone Camera URL", "Enter the phone stream
    URL (e.g., http://192.168.x.x:8080/video):")
    if ip:
        threading.Thread(target=run_video_capture, args=(ip,)).start()
```

Explanation

- This function asks the user to manually input the IP stream URL of their phone camera.
- Uses `simpdialog.askstring(...)` from the `tkinter` library to prompt the user with a text box.
 - For example, users might input: `http://192.168.1.10:8080/video`
 - This works with mobile apps like IP Webcam (Android) that stream camera footage over a local network.
- If the user inputs a valid URL:
 - `run_video_capture(ip)` is launched in a new thread.

Purpose of Threading

- Both `run_webcam()` and `run_phone_cam()` use threading to:
 - Run camera capture concurrently with the GUI.
 - Allow you to still interact with buttons (like "Save Results" or "Classify Image") while the video is streaming.

Summary

Function	Input Source	Runs In Thread?	Notes
<code>run_webcam()</code>	System webcam (0)	Yes	Starts the main camera with default device index
<code>run_phone_cam()</code>	IP webcam URL string	Yes	Prompts the user to enter the stream URL of their phone camera

Both connect to the same processing function, ensuring that YOLO and CNN models are applied the same way regardless of the camera source.

10. Saving Results

```
def save_to_excel():  
    ...  
    df.to_excel("lego_classification_results.xlsx", index=False)
```

- ♦ Filters results by confidence > 0.4 and saves as Excel.

11. Single Image Classification

```
def browse_and_classify_only():  
    ...
```

- ♦ Lets the user choose an image file.
- ♦ Classifies it using MobileNet only.
- ♦ Shows label, confidence, and displays the image in the GUI.

12. GUI Interface

```
root = tk.Tk()  
root.title("Brick Classifier GUI")
```

- ♦ Simple GUI with buttons:
 - Classify image (single).
 - Start webcam.
 - Start phone camera.
 - Save to Excel.
 - ♦ Also shows image preview and classification result.
-

Summary of Key Features:

Feature	Description
YOLO + CNN	Ensemble used intelligently with fallback.
MobileNet only classes	Some classes bypass YOLO for better accuracy.
Feature	Description
Color recognition	Matches to real LEGO colors.
Webcam & IP Cam	Toggle input source via GUI.
Excel Export	Save all results with label, confidence, and color.
GUI	Tkinter-based interface with live image display and result.

The End